Cut And Learn for Unsupervised Object Detection and Instance Segmentation

2023

이미지 처리팀

김병현 **안종식** 이주영 이해원 이희재

CONTENTS

01

Introduction

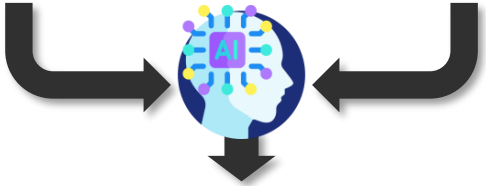
01. Introduction

1. Unsupervised Object Detection & Instance Segmentation

Supervised Learning

[Train Data]

[Annotations]

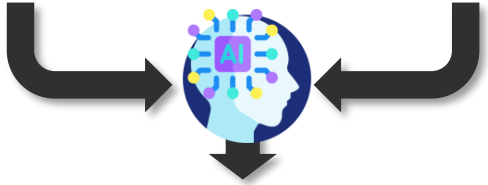


[Inference 결과]

Semi Supervised Learning

[Train Data]

[A Little Annotations]



[Inference 결과]

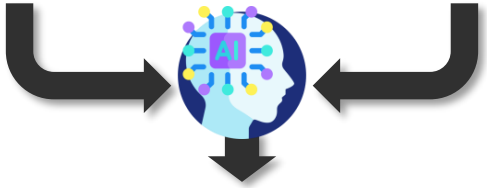
01. Introduction

1. Unsupervised Object Detection & Instance Segmentation

Unsupervised Learning

[Train Data]

[With Out Any Annotations]



[Inference 결과]

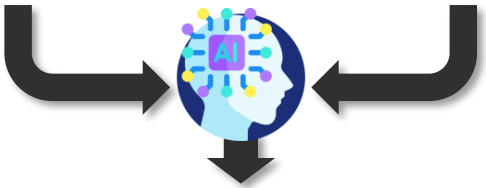
01. Introduction

1. Unsupervised Object Detection & Instance Segmentation

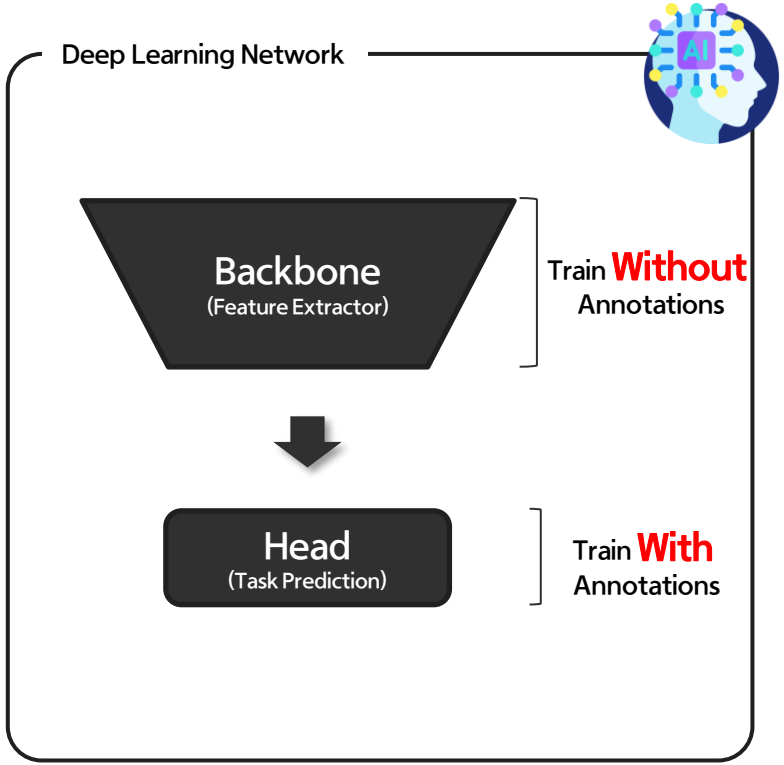
Self Supervised Learning

[Train Data]

[With Out Any Annotations]

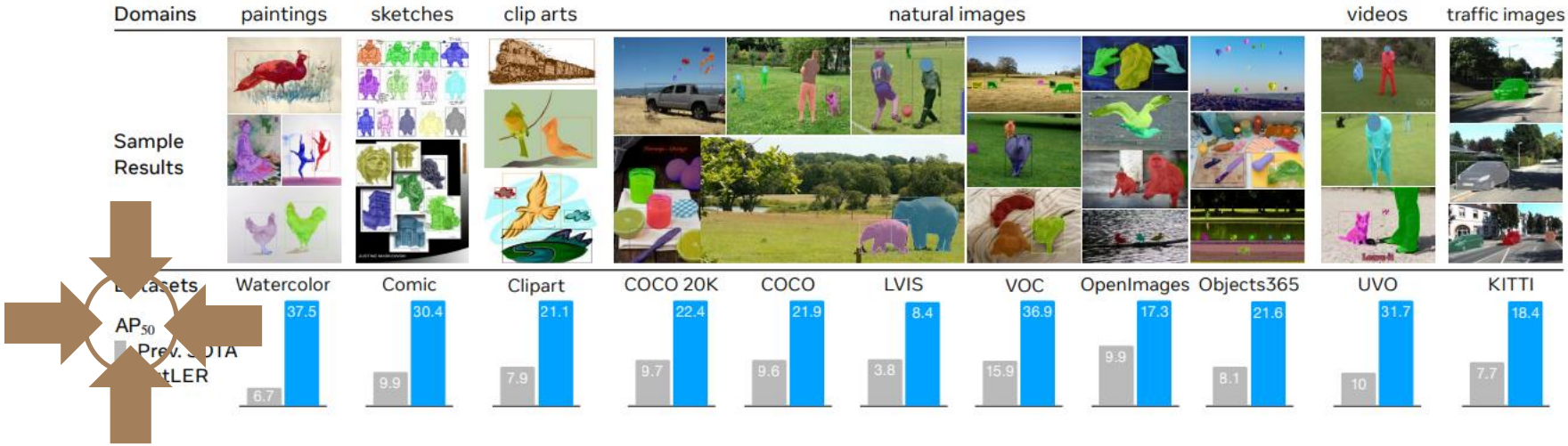


[Inference 결과]



01. Introduction

2. Class Agnostic Detection



Class Aware Detection

- w/ Supervision : 몇 개의 Class로 분류해야 하는지 알고 있음 (관심 객체가 무엇인지 알고 있음)

Class Agnostic Detection

- w/o Supervision : 몇 개의 Class로 분류해야 하는지 알 수 없음 (관심 객체가 무엇인지 인지 불가)

[Class Aware]

[Class Agnostic]

01. Introduction

3. CutLER (Cut-and-LEaRn)

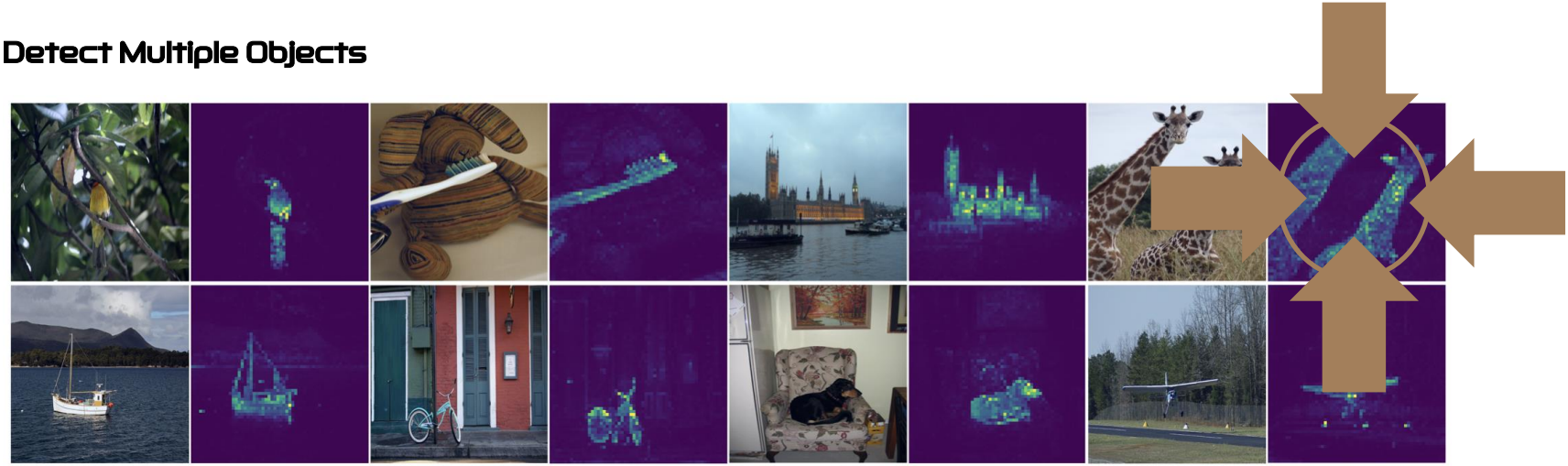
Contribution of CutLER

		Previous Works				
		DINO	LOST	TokenCut	FreeSOLO	Ours
①	detect multiple objects	✗	✓	✗	✓	✓
②	zero-shot detector	✓	✗	✓	✗	✓
③	compatible with various detection architectures	-	✓	-	✗	✓
④	pretrained model for supervised detection	✓	✗	✗	✓	✓

01. Introduction

3. CutLER (Cut-and-LEaRn)

① Detect Multiple Objects



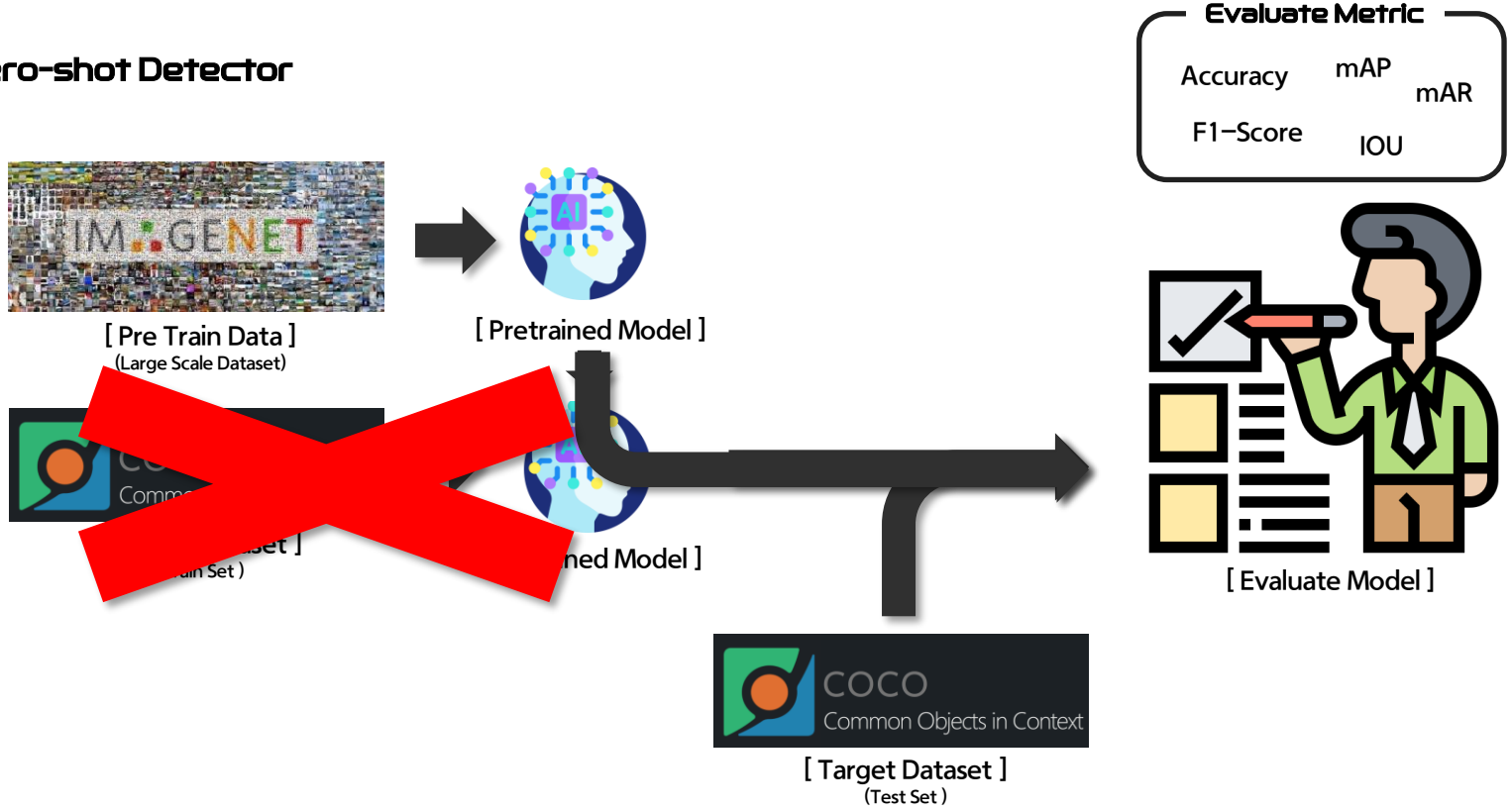
DINO : 한 개의 Object만 검출 가능

- 이미지 내 한 개의 Object 만 검출 가능 (제한적인 데이터셋에서 적용가능)
- 여러 Object가 있어도 Semantic Mask만 검출 가능 (Instance 단위로 분리 X)
- Self Supervised Learning을 통한 Feature Extractor 학습 방법이므로 실제 Target Task를 수행하는 방법이 아님

01. Introduction

3. CutLER (Cut-and-LEaRn)

② Zero-shot Detector

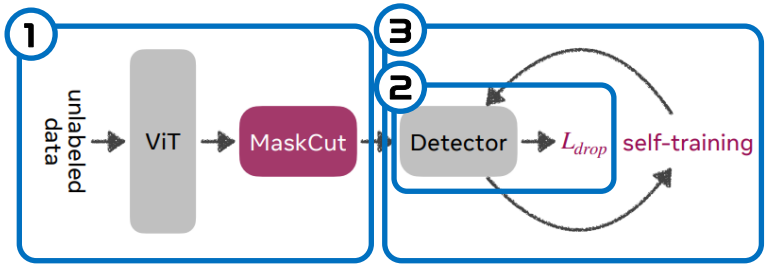


01. Introduction

3. CutLER (Cut-and-LEaRn)

③ Compatible with various detection architectures

Pipeline



① ③

전 / 후처리로 생각할 수 있어 다양한 Detector에 적용 가능

② 기존 Loss에 Indicator Function 추가

$$\mathcal{L}_{\text{drop}}(r_i) = \mathbb{1}(\text{IoU}_i^{\max} > \tau^{\text{IoU}})\mathcal{L}_{\text{vanilla}}(r_i)$$

- 어떠한 Detector도 Loss Term 수정을 통해 CutLER 적용 가능

② 다양한 구조 적용 예시 Ablation Section에서 확인 가능

	Mask R-CNN	Cascade Mask R-CNN	ViTDet
AP_{50}^{box} / AP^{box}	20.3 / 10.6	20.8 / 11.5	21.5 / 11.8
AP_{50}^{mask} / AP^{mask}	17.2 / 8.5	17.7 / 8.8	18.0 / 9.0

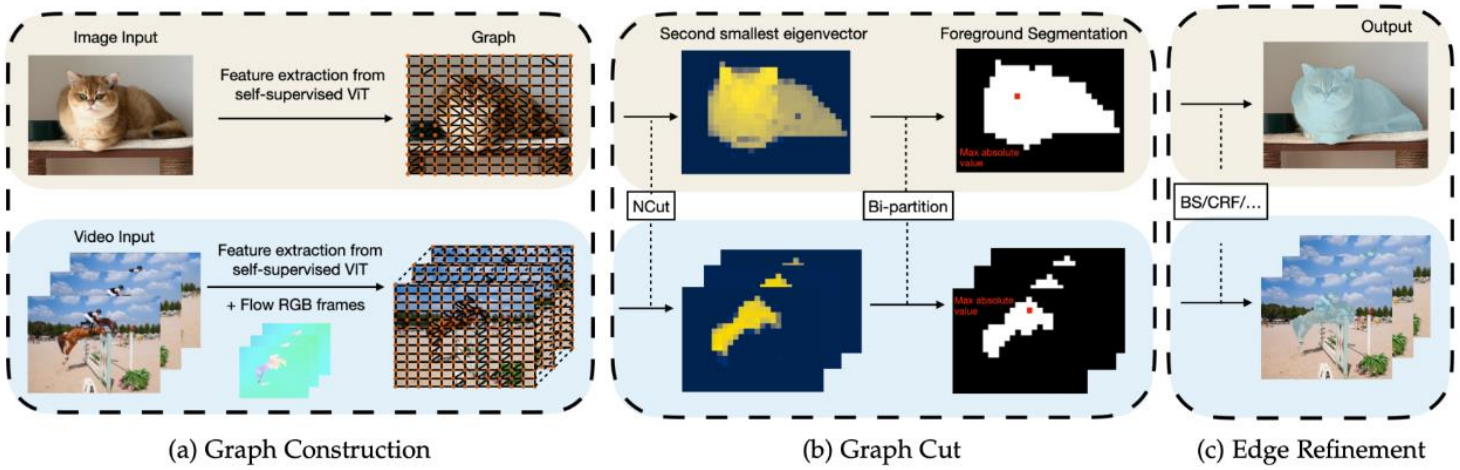
01. Introduction

3. CutLER (Cut-and-LEaRn)

④ Pretrained Model for Supervised Detection

Pretrained Model을 생성하기 위한 과정으로 활용 가능

- TokenCut의 경우, 엄밀히 말해 Deep Learning 기반의 Model이 아닌 Graph Cut의 알고리즘 중 한 종류이므로 Pretrain Model로서 활용 불가



02

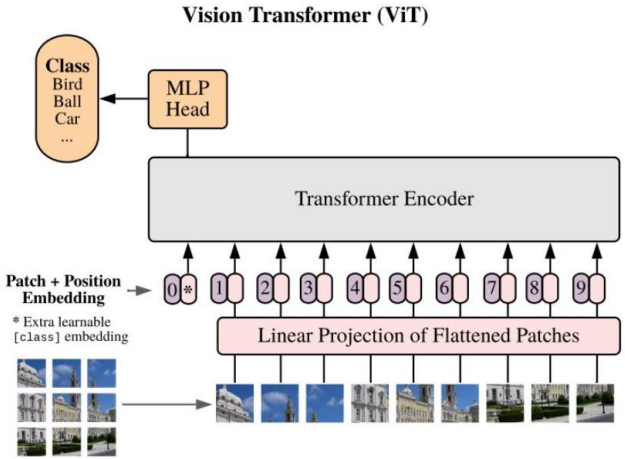
Related Works

02. Related Works

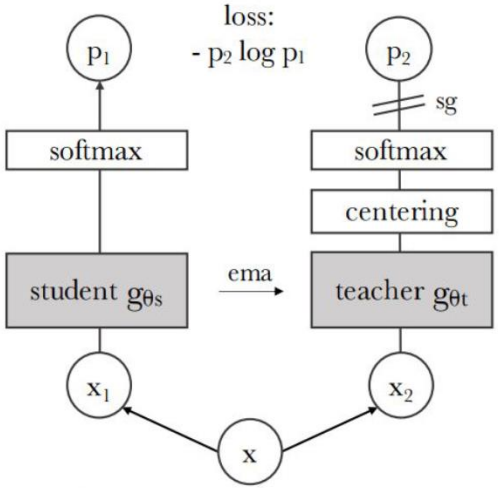
1. DINO (Emerging Properties in Self-Supervised Vision Transformers)

Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut

- 이미지처리팀 현청천, https://www.youtube.com/watch?v=JCEK5nD4MKM



[ViT Model]



[DINO (Self– Supervised Learning Method]

02. Related Works

2. TokenCut (Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut)

Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut

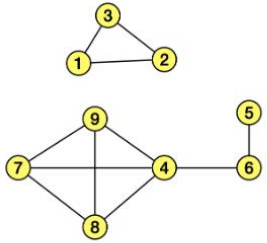
- 이미지처리팀 현청천, https://www.youtube.com/watch?v=JCEK5nD4MKM

	1	2	3	4	5	6	7	8	9
1	0	1	1	0	0	0	0	0	0
2	1	0	1	0	0	0	0	0	0
3	1	1	0	0	0	0	0	0	0
4	0	0	0	0	0	1	1	1	1
5	0	0	0	0	0	1	0	0	0
6	0	0	0	1	1	0	0	0	0
7	0	0	0	1	0	0	0	1	1
8	0	0	0	1	0	0	1	0	1
9	0	0	0	1	0	0	1	1	0

Adjacency Matrix

	1	2	3	4	5	6	7	8	9
1	2	0	0	0	0	0	0	0	0
2	0	2	0	0	0	0	0	0	0
3	0	0	2	0	0	0	0	0	0
4	0	0	0	4	0	0	0	0	0
5	0	0	0	0	1	0	0	0	0
6	0	0	0	0	0	2	0	0	0
7	0	0	0	0	0	0	3	0	0
8	0	0	0	0	0	0	0	3	0
9	0	0	0	0	0	0	0	0	3

Degree Matrix



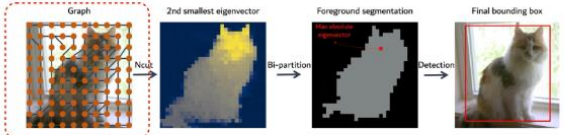


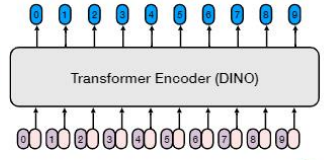
Figure 2: An overview of the TokenCut approach. We construct a graph where the nodes are tokens and the edges are similarities between the tokens using transformer features. The foreground and background segmentation can be solved by Ncut [43]. Performing bi-partition on the second smallest eigenvector allows to detect foreground object.

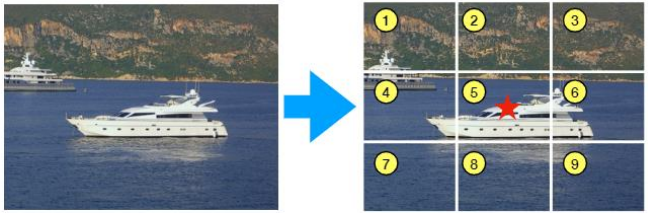
$$\mathcal{E}_{i,j} = \begin{cases} 1, & \text{if } S(v_i, v_j) \geq \tau \\ \epsilon, & \text{else} \end{cases}$$

$$S(v_i, v_j) = \frac{v_i v_j}{\|v_i\|_2 \|v_j\|_2}.$$

τ: 0.2

ϵ: $1e-5$



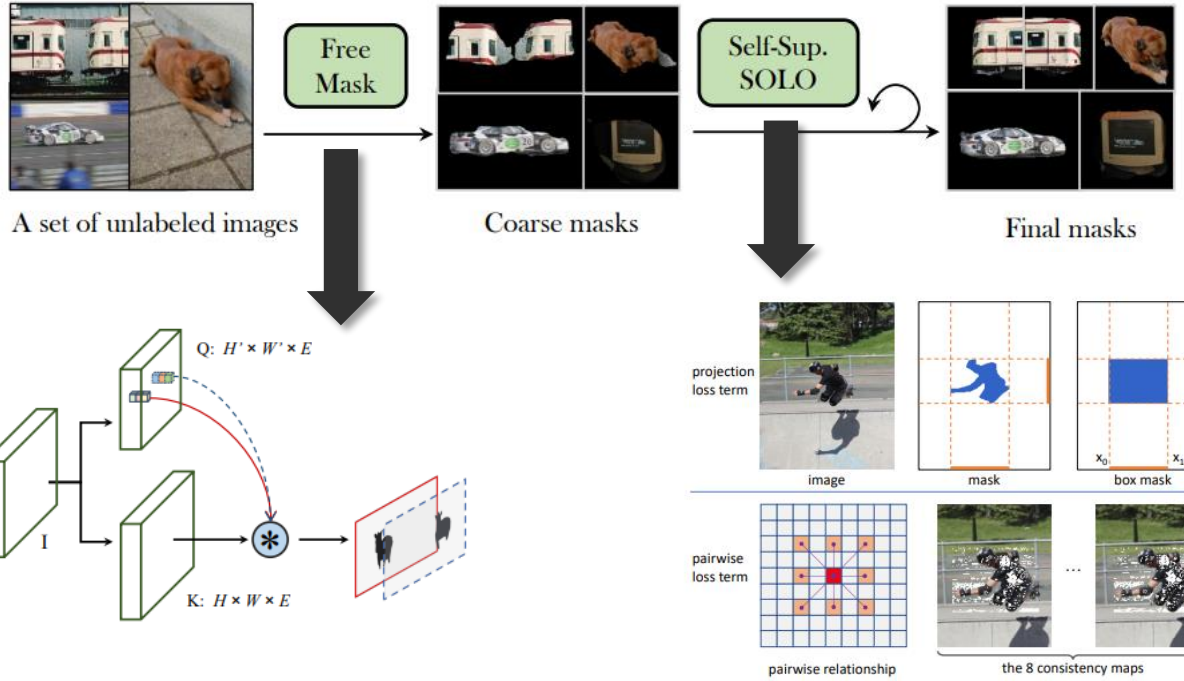


02. Related Works

3. FreeSOLO for unsupervised instance segmentation

Instance Segmentation Network인 SOLOv2에 Unsupervised Learning Method 적용

- SOLO 구조를 이용하여 Network Dependency가 존재 (다양한 Detector에 적용불가)



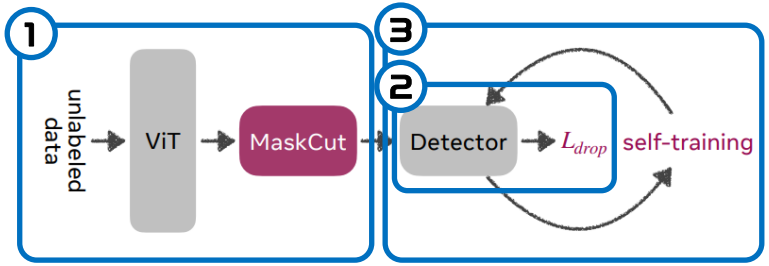
Q & A

03

Methods

03. Methods

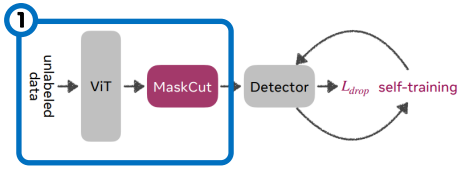
1. MaskCut for Discovering Multiple Objects



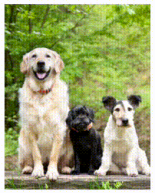
1. MaskCut for Discovering Multiple Objects
2. DropLoss for Exploring Image Regions
3. Multi-Round Self-Training

03. Methods

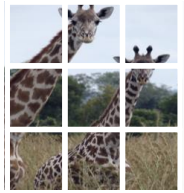
1. MaskCut for Discovering Multiple Objects



$n \times n$

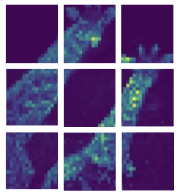


patchified input



N x N Patch Input Images

Self-Supervised ViT Model
(Trained By. DINO)

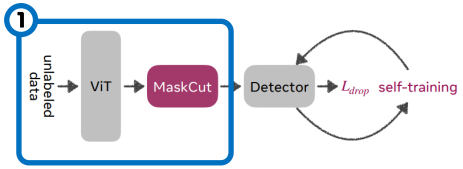


ViT Feature Vector

MaskCut

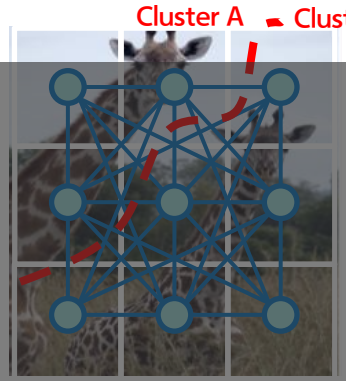
x N

03. Methods



1. MaskCut for Discovering Multiple Objects

NormalizedCut & TokenCut



: Graph Node

: Graph Edge

- Graph Edge i 는 다른 Graph Edge j와 가중치 ω_{ij}로 연결되어 있다.

  > 이를 표현한 행렬 : Adjacency Matrix (or Affinity Matrix), 인접 행렬

$$W = \begin{bmatrix} \omega_{00} & \cdots & \omega_{09} \\ \vdots & & \vdots \\ \omega_{90} & \cdots & \omega_{99} \end{bmatrix}$$

3x3의 Patch 존재하므로, 9 x 9의 Adjacency Matrix 생성

$= \frac{K_i K_j}{\quad}$ (Cosine Similarity)

- Weight Threshold τ_{ncut}적용, 1 또는 $1e^{-5}$로 Thresholding

NP HARD

(Nondeterministic polynomial)

- 적절하게 두개의 Cluster A,B를 찾는 문제 (Normalized Cut)

  $Cut(A,B) = \sum_{i \subseteq A, j \subseteq B} \omega_{ij}$: : Cluster A와 B사이 Node들의 가중치 합

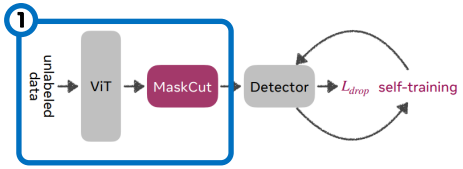
  $Vol(A)$: Cluster A 내부 Node끼리의 가중치 합

  ➡ $Argmin\big(NormalizedCut(A)\big) = Argmin(Cut(A,B) \cdot (\frac{1}{Vol(A)} + \frac{1}{Vol(B)}))$

  > 각기 다른 Cluster들끼리의 가중치 합을 최소로 만든다 !

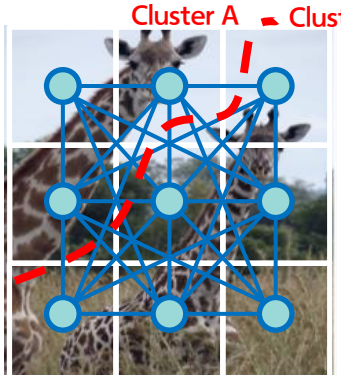
  > 각기 Cluster 내부의 가중치로 Normalization하여 군집내 유사도를 최대화 한다 !

03. Methods



1. MaskCut for Discovering Multiple Objects

NormalizedCut & TokenCut



: Graph Node

: Graph Edge

- Generalized Eigenvalue System으로 근사하여 최적의 Cluster를 계산

  > Laplacian Matrix를 이용 : Adjacency Matrix와 Degree Matrix를 이용하여 Laplacian Matrix 생성

$$D = \begin{bmatrix} d_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & d_9 \end{bmatrix}$$

- $d_i = \sum_{j=1}^{n} \omega_{ij}$
- 대각 성분을 제외한 나머지 값 = 0 : Diagonal Matrix
- Symmetric Matrix

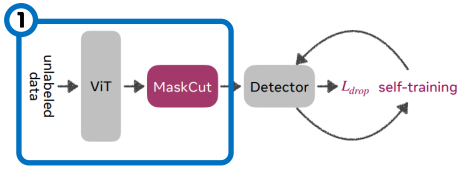
- 2nd Smallest Eigenvector 를 찾는 것이 $Argmin(NormalizedCut(A))$ 와 동일함[1)]

$$(D - W)x = \lambda Dx \ (\ x : Eigenvector, \lambda : Eigenvalue)$$

- 1st Smallest Eigenvector는 x의 Components가 모두 1인 Vector, $\lambda = 0$ 인 상황이므로 2nd Smallest Eigenvector를 계산

1) https://gjkoplik.github.io/spectral_clustering/#proofs_with_2_clusters

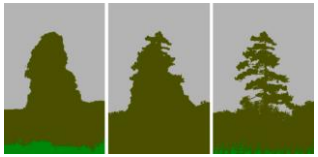
03. Methods



1. MaskCut for Discovering Multiple Objects

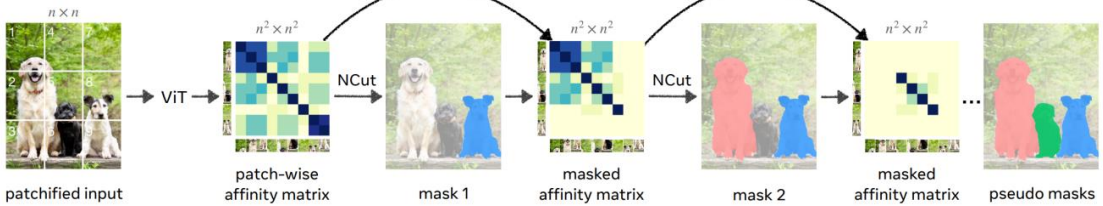
MaskCut

- 계산된 Cluster A, B의 Eigenvector에서, Absolute Value가 최대인 곳을 포함하는 Cluster를 Foreground로 채택
- Post Processing으로 DenseCRF (Conditional Random Field) 적용

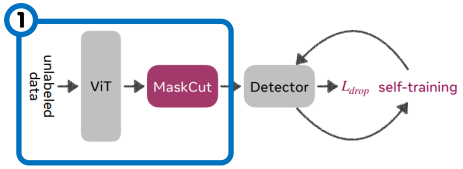


- t 단계에서 획득된 Foreground Mask를 제외한 Background Mask를 통해 Adjacency Matrix를 Masking하고 반복

$$W_{ij}^{t+1} = \frac{(K_i \blacksquare)(K_j \blacksquare)}{\|K_i\|_2 \|K_j\|_2}$$



03. Methods

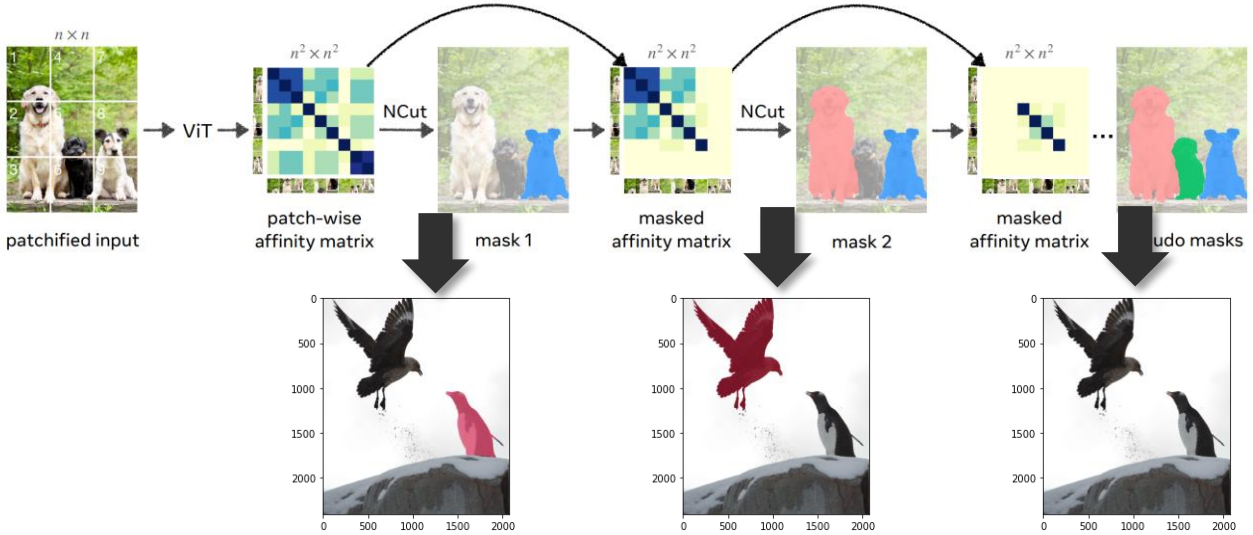


1. MaskCut for Discovering Multiple Objects

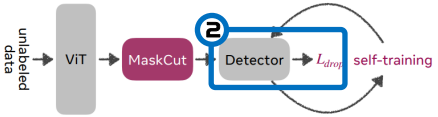
MaskCut

- Ablation을 통해 반복횟수 3이 제일 적절하다고 판단
- 실제 객체 갯수가 반복 횟수 3보다 적다면 ?

  > Weight Threshold τ_{ncut}에 의해 Mask 검출 X



03. Methods



2. DropLoss for Exploring Image Regions

MaskCut

- 실제 객체 갯수가 반복 횟수 3보다 많다면 ?

  > 검출이 안되는 객체 발생 !

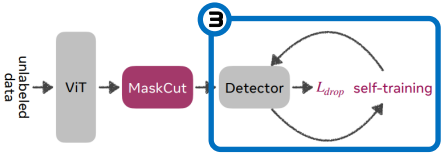
DropLoss

$$\mathcal{L}_{\text{drop}}(r_i) = \mathbb{1}(\text{IoU}_i^{\max} > \tau^{\text{IoU}})\mathcal{L}_{\text{vanilla}}(r_i)$$

- 기존 Loss를 그대로 사용할 경우, MaskCut이 찾지 못한 객체들을 찾지 못하도록 Detector가 학습됨
- MaskCut은 Coarse Mask이기 때문에 이러한 Coarse Mask와 겹치는 부분만 Loss를 계산하여 Detector가 새로운 객체를 찾아낼 수 있도록 함
- $\tau_{ncut} = 0.01$

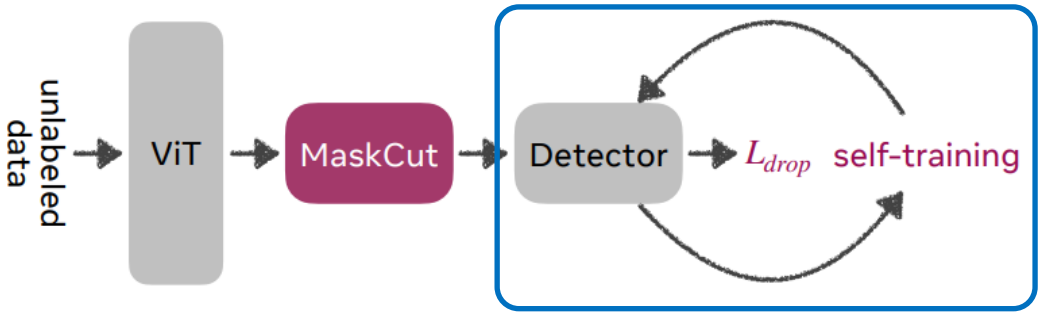
03. Methods

3. Multi-Round Self-Training



Self-Training

- 첫 단계 MaskCut으로 생성한 Coarse Mask 사용하여 학습 후에는, 이전 단계 Detector에서 검출한 Mask로 학습 반복 진행



Q & A

04

Experiments

04. Experiments

1. Results

Unsupervised Zero-shot Evaluation

Datasets →	Avg.		COCO		COCO20K		VOC		LVIS		UVO		Clipart		Comic		Watercolor		KITTI		Objects365		OpenImages	
Metrics →	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR	AP_{50}	AR
Prev. SOTA [47]	9.0	13.4	9.6	12.6	9.7	12.6	15.9	21.3	3.8	6.4	10.0	14.2	7.9	15.1	9.9	16.3	6.7	16.2	7.7	7.1	8.1	10.2	9.9	14.9
CutLER	24.3	35.5	21.9	32.7	22.4	33.1	36.9	44.3	8.4	21.8	31.7	42.8	21.1	41.3	30.4	38.6	37.5	44.6	18.4	27.5	21.6	34.2	17.3	29.6
vs. prev. SOTA	**+15.3**	**+22.1**	**+12.3**	**+20.1**	**+12.7**	**+20.5**	**+21.0**	**+23.0**	**+4.6**	**+15.4**	**+21.7**	**+28.6**	**+13.2**	**+26.2**	**+20.5**	**+22.3**	**+30.8**	**+28.4**	**+10.7**	**+20.4**	**+13.5**	**+24.0**	**+7.4**	**+14.7**

- Imagenet에서 학습한 Model을 각기 Dataset에 적용하였을 때 (w/o Finetuning) 기존 방법론 대비 성능 2배~4배 향상 확인
- 이 때, FreeSOLO는 Resnet101 Backbone인데 반해, CutLER은 Resnet50

Methods	Pretrain	Detector	Init.	COCO 20K						COCO val2017					
				AP^{box}_{50}	AP^{box}_{75}	AP^{box}	AP^{mask}_{50}	AP^{mask}_{75}	AP^{mask}	AP^{box}_{50}	AP^{box}_{75}	AP^{box}	AP^{mask}_{50}	AP^{mask}_{75}	AP^{mask}
non zero-shot methods															
LOST [38]	IN+COCO	FRCNN	DINO	-	-	-	2.4	1.0	1.1	-	-	-	-	-	-
MaskDistill [42]	IN+COCO	MRCNN	MoCo	-	-	-	6.8	2.1	2.9	-	-	-	-	-	-
FreeSOLO* [47]	IN+COCO	SOLOv2	DenseCL	9.7	3.2	4.1	9.7	3.4	4.3	9.6	3.1	4.2	9.4	3.3	4.3
zero-shot methods															
DETReg [3]	IN	DDETR	SwAV	-	-	-	-	-	-	3.1	0.6	1.0	8.8	1.9	3.3
DINO [7]	IN	-	DINO	1.7	0.1	0.3	-	-	-	-	-	-	-	-	-
TokenCut [50]	IN	-	DINO	-	-	-	-	-	-	5.8	2.8	3.0	4.8	1.9	2.4
CutLER (ours)	IN	MRCNN	DINO	21.8	11.1	10.1	18.6	9.0	8.0	21.3	11.1	10.2	18.0	8.9	7.9
CutLER (ours)	IN	Cascade	DINO	22.4	12.5	11.9	19.6	10.0	9.2	21.9	11.8	12.3	18.9	9.7	9.2
vs. prev. SOTA				**+12.7**	**+9.3**	**+7.8**	**+9.9**	**+6.6**	**+4.9**	**+12.3**	**+8.7**	**+8.1**	**+9.5**	**+6.4**	**+4.9**

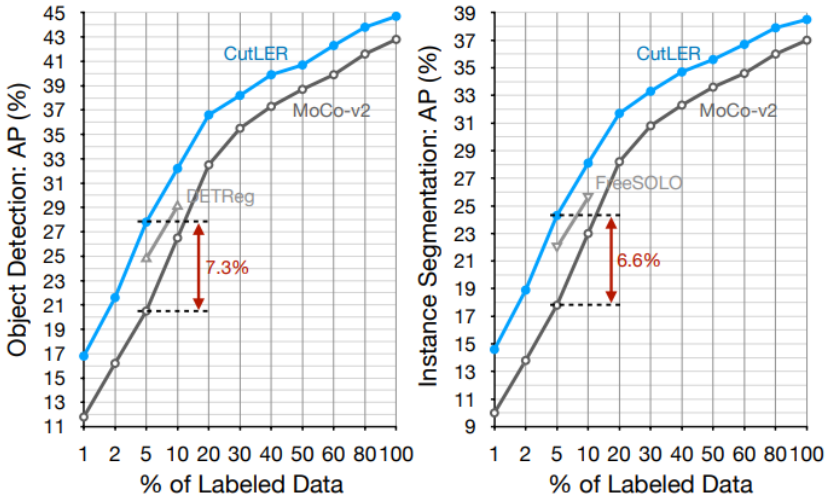
- Non zero-shot Methods와 비교하여도 성능 향상 확인 가능

04. Experiments

1. Results

Label-Efficient and Fully-Supervised Learning

- Pretrain Model로서 성능 확인을 위해서 Fully Annotated Dataset을 Subset으로 나누어 학습 및 평가 진행



- MoCo-v2 : Self-Supervised를 통한 Pretrain Model 생성 방법
- Self-Supervised 방법론을 사용한 방법보다 성능 우수

04. Experiments

2. Ablation

각 Component 별 성능 향상

- Pretrain Model로서 성능 확인을 위해서 Fully Annotated Dataset을 Subset으로 나누어 학습 및 평가 진행

Methods	UVO		COCO	
	AP^{mask}_{50}	AP^{mask}	AP^{mask}_{50}	AP^{mask}
TokenCut [50]	-	-	4.9	2.0
Base	14.6	5.4	13.5	5.7
+ MaskCut	19.3	8.1	15.8	7.7
+ DropLoss	20.9	9.0	16.6	8.2
+ copy-paste [16, 19]	21.5	9.9	17.7	8.8
+ self-train (CutLER)	22.8	10.1	18.9	9.7

Coarse Mask 생성방식에 따른 결과

Methods	AP^{box}_{50}	AP^{box}	AR^{box}_{100}	AP^{mask}_{50}	AP^{mask}	AR^{mask}_{100}
TokenCut (1 eigenvec.)	5.2	2.6	5.0	4.9	2.0	4.4
TokenCut (3 eigenvec.)	4.7	1.7	8.1	3.6	1.2	6.9
MaskCut ($t = 3$)	6.0	2.9	8.1	4.9	2.2	6.9
CutLER	21.9	12.3	32.7	18.9	9.7	27.1

04. Experiments

2. Ablation

Hyper Parameters

Size →	240	360	480	640
AP_{50}^{mask}	15.1	16.6	17.7	17.9

(a) Image size.

τ^{ncut} →	0	0.1	0.15	0.2	0.3
AP_{50}^{mask}	17.1	17.5	17.7	17.6	17.5

(b) τ^{ncut} for MaskCut.

N →	2	3	4
AP_{50}^{mask}	16.9	17.7	17.7

(c) # masks per image.

τ^{IoU} →	0	0.01	0.1	0.2
AP_{50}^{mask}	17.4	17.7	14.4	12.7

(d) τ^{IoU} for DropLoss.

Self-Train 횟수에 따른 성능

	UVO			COCO		
	AP_{50}^{mask}	AP^{mask}	AP_{75}^{mask}	AP_{50}^{mask}	AP^{mask}	AP_{75}^{mask}
1 round	20.6	9.0	7.0	17.7	8.8	8.0
2 rounds	22.2	9.6	7.5	18.5	9.5	8.8
3 rounds	22.8	10.1	8.0	18.9	9.7	9.2
4 rounds	22.8	10.2	8.2	18.9	9.8	9.3

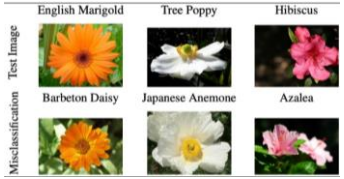
05

Conclusion

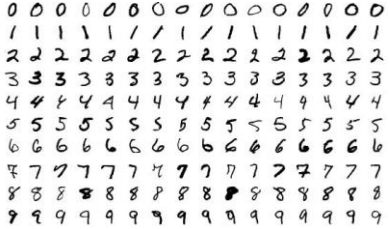
05. Conclusion

Unsupervised Learning 방법론 또한 General한 Dataset에서 활용되기 시작하였다.

- 한정적인 상황, 하나의 객체 위주 등 제약사항이 많았지만 일반적인 데이터셋에서도 사용할 수 있게 발전하기 시작



[Oxford 102 Flowers]



[MNIST]

- 발전의 원인 : DINO와 같은 Transformer 기반의 Self-Supervised 기법이 촉매제가 되었을 것

Class-Agnostic이라는 한계는 존재

- One Class Object Detection & Instance Segmentation

Q & A

Thank you for your attention